

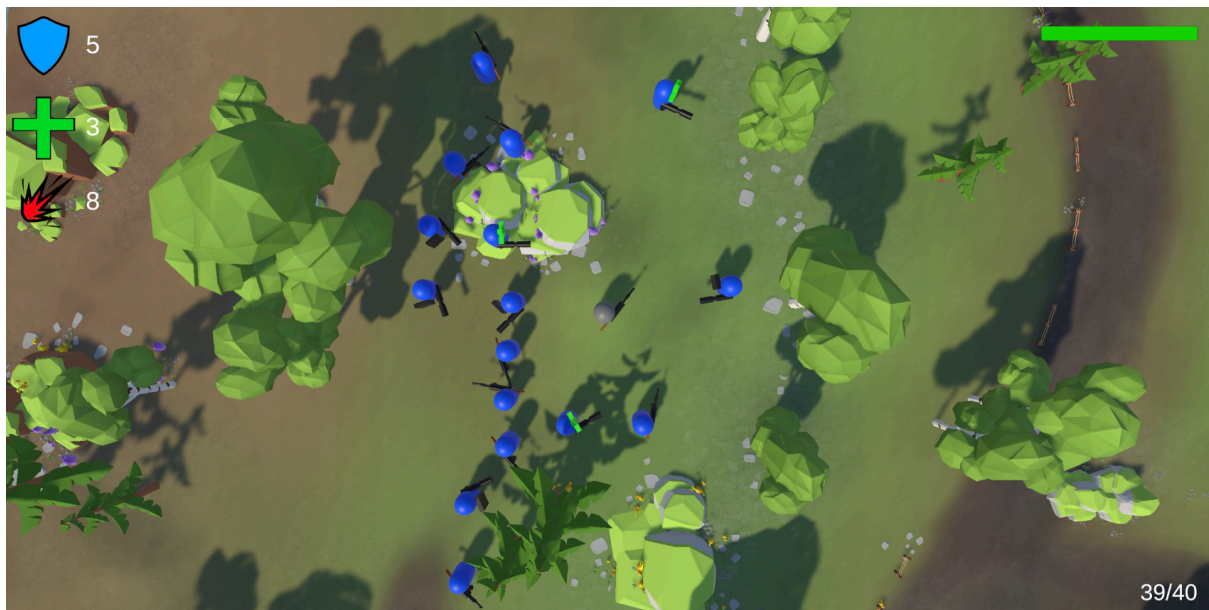
Utilisation :

Input :

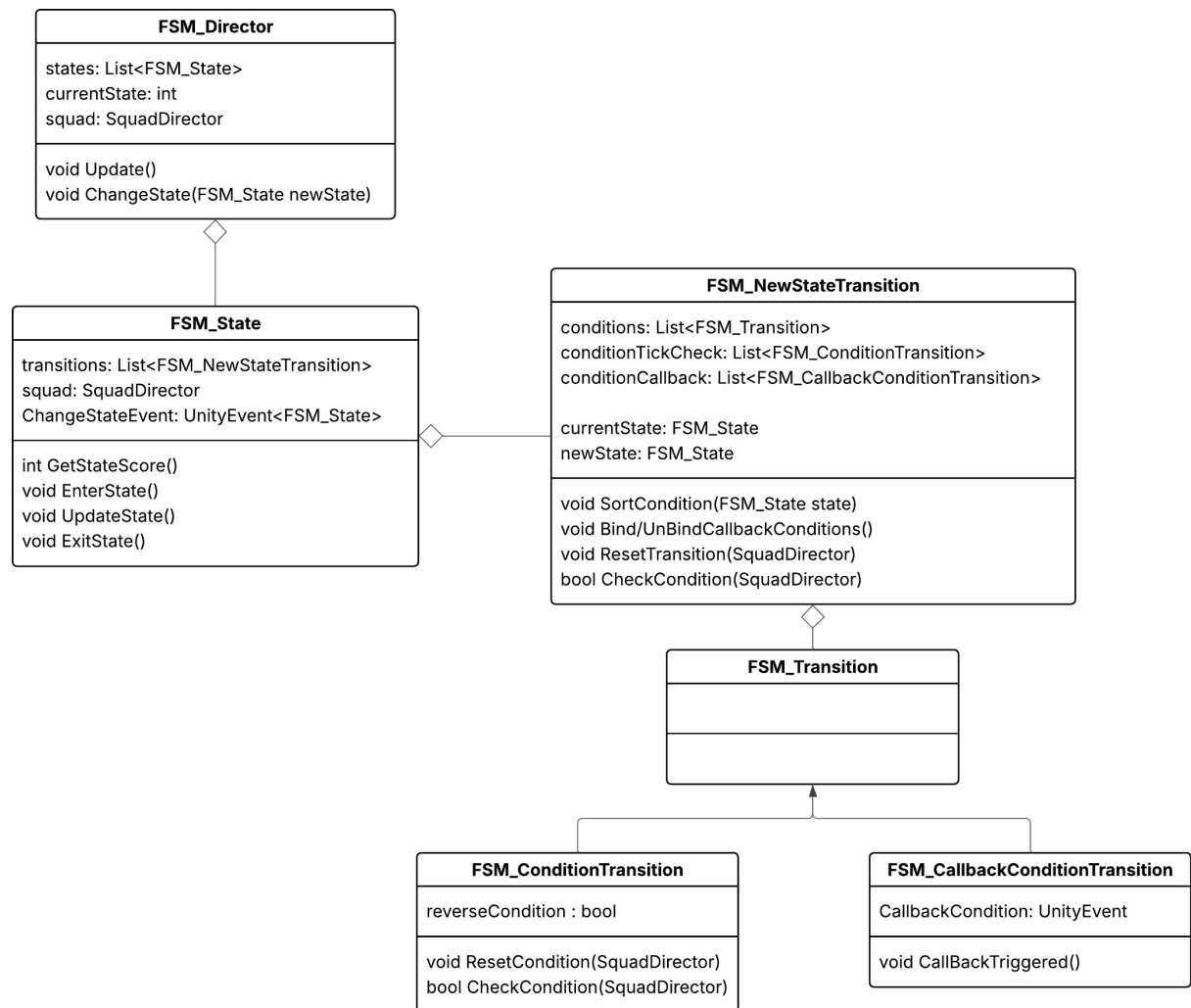
- ZQSD/WASD : déplacement
- Clique Gauche : tire du joueur
- Clique Droit : demande de cover du joueur
- Tab : ouvre le menu

Le joueur apparaît avec une équipe d'ia et peut se déplacer dans la carte pour rencontrer des groupes d'ia agressifs.

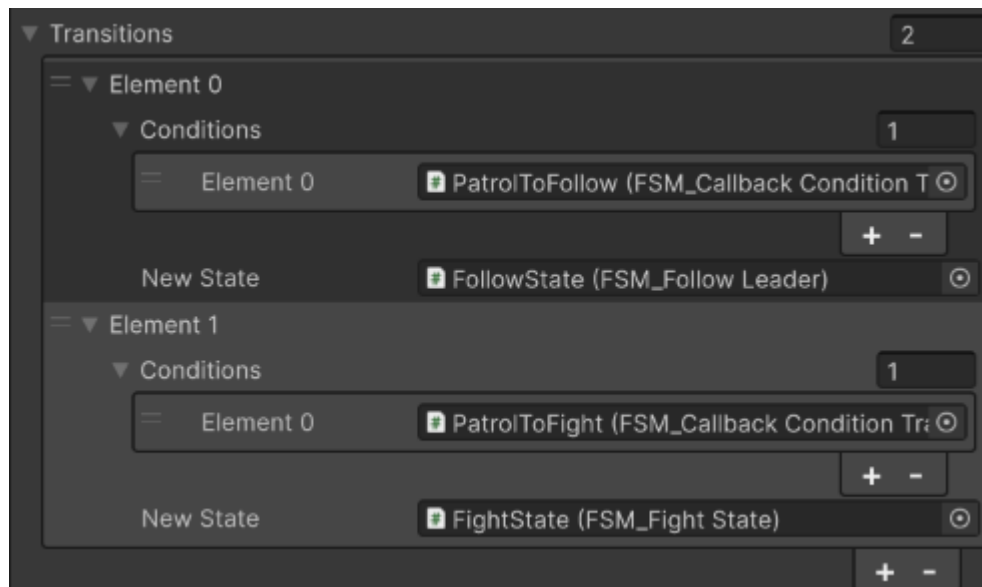
Dans la console de debug est affiché les changements de states des FSM de squad.



Le mouvement :



Pour les mouvements de l'escouade, nous avons décidé d'utiliser une **FSM**. Dans l'éditeur, les différents states ainsi que leurs transitions apparaissent en enfants de l'escouade.



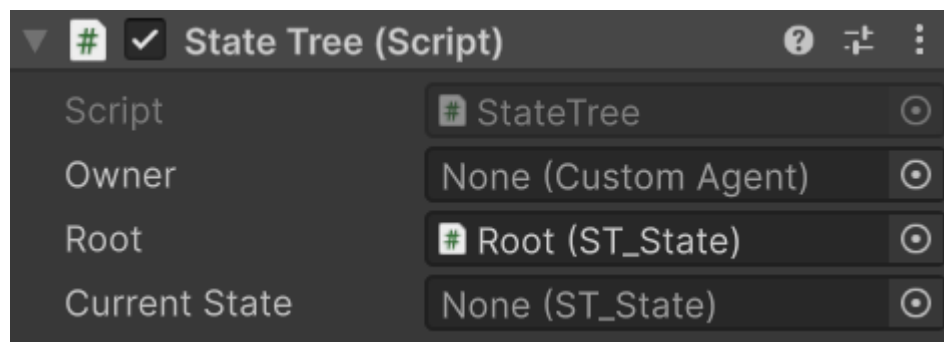
Pour qu'une transition soit valide, toutes les conditions héritant de **FSM_ConditionTransition** doivent être validées, ou bien une condition de type **FSM_CallbackConditionTransition** doit être trigger.

Si plusieurs transitions sont valides simultanément, un système de score a été implémenté afin de déterminer la priorité. Cependant, celui-ci n'a finalement pas été utilisé, car le nombre limité de transitions différentes ne justifiait pas réellement son intérêt.

Les **FSM_ConditionTransition** sont vérifiées dans l'update du state, tandis que les **FSM_CallbackConditionTransition** sont bind et unbind au start et à l'exit du state.

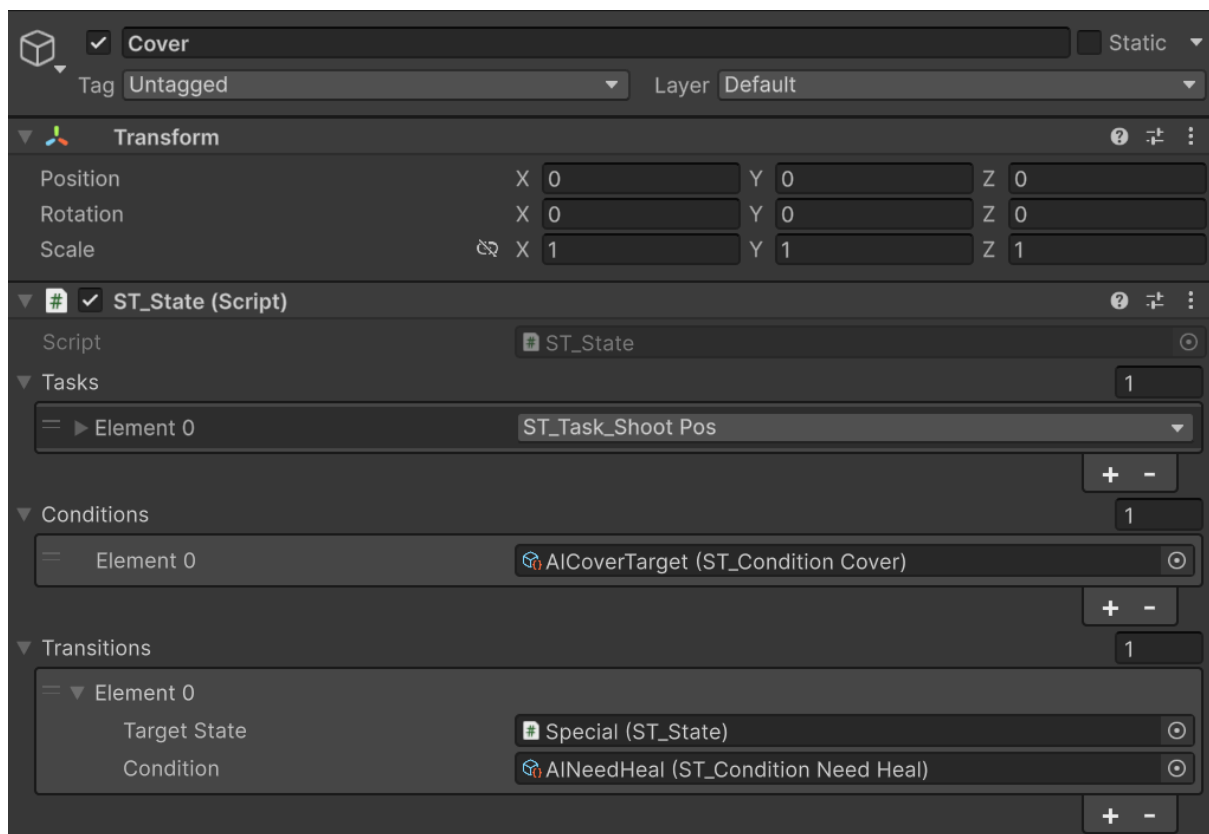
Le comportement :

Pour le comportement, nous avons opté pour un state tree.



Le state tree connaît son agent, sa root et son état courant.

Chaque states contient une liste de tâches à effectuer, de conditions d'entrée et de transitions donnant la condition de transition et l'état vers lequel transitionner.



Pour parcourir le state tree, l'objet state tree entre dans le state root.

Le state root va ensuite regarder ses tasks et s'il n'en a pas, il va regarder dans les états suivants et vérifier les conditions d'entrée de chaque état.

Dès qu'il trouve un état dans lequel il peut entrer, il définit cet état comme étant l'état courant et entrer dans cet état.

Ces actions vont se répéter jusqu'à trouver un état qui n'a pas de suivant ou d'un état qui possède une task.

Si l'état courant possède une task, elle est jouée et l'état reste l'état courant tant que toutes les tâches ne sont pas considérées comme fini.

Quand le state tree sort d'un état sans état suivant, le dernier état joué appelle une fonction du state tree qui relance la recherche d'état à la prochaine boucle d'update pour que la recherche d'état ne bloque pas le thread principal d'unity.